

Performance Modeling and Analysis: Roofline Model Evaluation of Neural Networks

COMS 6998: Applied Machine Learning in Cloud — Project 1

Geonsik Moon (gm3239)

March 27, 2026

A Experiment Design

The objective of this experiment is to evaluate and compare the computational bottlenecks of different neural network architectures across multiple generations of NVIDIA GPU hardware. Our central hypothesis is that the dominant performance bottleneck (memory-bound vs. compute-bound) is determined by a combination of the model’s intrinsic algorithmic properties, specifically its ratio of floating-point operations to memory accesses, and the hardware’s balance between compute throughput and memory bandwidth. We expect that attention-based models like the Vision Transformer (ViT), which rely on dense matrix multiplications, will exhibit a higher Arithmetic Intensity (AI) and tend toward the compute-bound region, while convolutional models (ResNet, VGG), whose many small-kernel convolutions generate proportionally more memory traffic, will remain closer to or within the memory-bound region.

A.1 Environments

To observe how hardware scaling affects performance placement on the Roofline model, we evaluate three distinct GPU environments provided via Google Colab Pro:

- **NVIDIA T4** (Turing Architecture, 2018): 8.1 TFLOPS FP32 peak compute, 320 GB/s memory bandwidth. Ridge Point ≈ 25.31 FLOPs/Byte.
- **NVIDIA L4** (Ada Lovelace Architecture, 2023): 30.3 TFLOPS FP32 peak compute (with TF32 Tensor Core), 300 GB/s memory bandwidth. Ridge Point ≈ 101.0 FLOPs/Byte.
- **NVIDIA A100** (Ampere Architecture, 2020): 19.5 TFLOPS FP32 peak compute, 1,555 GB/s memory bandwidth. Ridge Point ≈ 12.54 FLOPs/Byte.

These three GPUs represent distinct points in NVIDIA’s generational progression, providing a broad spectrum of compute-to-bandwidth ratios that stress different aspects of neural network workloads.

A.2 Neural Network Models

We profile the following three architectures on dummy ImageNet-scale data ($224 \times 224 \times 3$ images, batch size $B = 8$):

- **ResNet-50**: A 50-layer residual CNN with skip connections. Its many small (3×3) convolutions produce a moderate parameter count (~ 25.6 M) but a large activation footprint.
- **VGG-16**: A 16-layer CNN with a uniform 3×3 convolutional design and large fully-connected layers. The largest model in our study (~ 138.4 M parameters), generating very high memory traffic.
- **Vision Transformer (ViT-B-16)**: A transformer-based architecture that partitions images into 16×16 patches and processes them through 12 multi-head self-attention encoder blocks. Its dominant computation pattern is large, dense matrix multiplications from the attention mechanism.

A.3 Profiling Methodology

Each model was profiled in both **training** (forward + backward + optimizer step) and **evaluation** (forward-only) modes. Data was collected using NVIDIA Nsight Compute (`ncu`) on a single profiled batch. To avoid cold-start anomalies, we injected `torch.cuda.profiler` hooks to profile only the 11th batch after a 10-batch warm-up phase. Dummy data was used since our focus is on *application characteristics and system resource usage*, not model accuracy.

Limitation: Our measurements are based on a single profiled batch per configuration (model $\times$ mode $\times$ GPU). While Nsight Compute’s kernel-level instrumentation is deterministic for a given execution, batch-to-batch variance in cuDNN algorithm selection, GPU clock throttling, and memory allocation patterns could introduce variability. We mitigate this through the warm-up phase, but acknowledging this single-sample limitation is important since multi-batch averaging would strengthen the statistical confidence of the results.

B Complexity Estimation

Using the `torchsummary` (for CNNs) and `torchinfo` (for ViT) frameworks, we analytically estimated the parameter count, the forward/backward pass memory footprint, and the total estimated memory footprint for each architecture. These estimates characterize each model’s static memory demands and provide context for interpreting the empirical DRAM traffic observed during profiling.

B.1 Memory Complexity

Model	Parameters (M)	Fwd/Bwd Size (MB)	Params Size (MB)	Est. Total (MB)
ResNet-50	25.6	286.57	97.49	384.63
VGG-16	138.4	218.97	527.80	747.34
ViT-B-16	86.6	104.09	232.27	336.96

Table 1: Memory Complexity Estimation per Architecture (input size: $1 \times 3 \times 224 \times 224$)

Several observations emerge from the memory complexity analysis:

1. **VGG-16 has the highest total memory footprint** (747.34 MB) due to its three massive fully-connected layers, which alone contribute over 120M of its 138.4M parameters. This predicts high DRAM traffic during profiling.
2. **ResNet-50 has the largest forward/backward pass size** (286.57 MB) despite having the fewest parameters. This is because residual connections require storing intermediate activations at every skip-connection boundary for gradient computation during backpropagation.
3. **ViT-B-16 has the most compact activation footprint** (104.09 MB) because its patch-embedding scheme reduces the spatial dimensions early (from 224×224 to 14×14 patches), and subsequent transformer blocks operate on a fixed sequence length of 197 tokens with 768-dimensional embeddings.

B.2 Computational Complexity (Analytical FLOPs)

Before measuring hardware counters, we derive analytical forward-pass FLOPs estimates for each architecture to establish baseline expectations. Each MAC (Multiply-Accumulate) counts as 2 FLOPs.

Model	Forward FLOPs (GFLOPs)	Derivation Summary
ResNet-50	~ 4.1	$2 \times$ MACs from 53 conv. layers + 1 FC layer
VGG-16	~ 15.5	$2 \times$ MACs from 13 conv. layers + 3 FC layers
ViT-B-16	~ 17.6	12 encoder blocks $\times$ (4 attention proj. + 2 MLP proj.)

Table 2: Analytical Forward-Pass FLOPs per Architecture ($B = 1$, input: $224 \times 224 \times 3$)

For a **single-image forward pass**, ResNet-50 requires ~ 4.1 GFLOPs (dominated by 3×3 convolutions across its bottleneck blocks), VGG-16 requires ~ 15.5 GFLOPs (with ~ 3 GFLOPs from its three FC layers alone), and ViT-B-16 requires ~ 17.6 GFLOPs (primarily from 12 layers of four attention projections at 197×768 each, plus two MLP projections at 768×3072).

Scaling to our profiling configuration ($B = 8$, training mode with backward pass $\approx 2 \times$ forward FLOPs), we can derive rough expected total FLOPs:

- ResNet-50 Training: $4.1 \times 8 \times 3 \approx 98$ GFLOPs
- VGG-16 Training: $15.5 \times 8 \times 3 \approx 372$ GFLOPs
- ViT-B-16 Training: $17.6 \times 8 \times 3 \approx 422$ GFLOPs

These back-of-envelope estimates serve as critical sanity checks against the empirical measurements. As we will see in Section C, the T4 measurements (173.7B, 347.3B, 889.6B FLOPs for ResNet-50, VGG-16, and ViT respectively) exceed these estimates by $1.8\text{--}2.1 \times$. This discrepancy is expected because our analytical estimate uses a simple $3 \times$ multiplier for training (fwd + bwd + optimizer), whereas in practice, cuDNN algorithm selection (e.g., Winograd transforms), additional memory management kernels, and gradient accumulation contribute extra operations. Critically, the T4 measurements remain within the same order of magnitude as the analytical predictions, lending confidence to their accuracy. In contrast, A100 CNN measurements (7.1B and 12.6B FLOPs for ResNet-50 and VGG-16) fall $14\text{--}30 \times$ *below* the analytical predictions, immediately flagging a profiling anomaly that we investigate in Section E.

B.3 Predicted Arithmetic Intensity

Using the analytical forward FLOPs and the memory footprint data, we can estimate a rough *expected* AI before measurement. For a training run, the total memory traffic includes reading activations, weights, and gradients, plus writing updated weights. As a first approximation, training DRAM traffic scales with the total estimated memory size multiplied by a factor accounting for forward read, backward read, and gradient write. Using the T4’s bandwidth (320 GB/s) and ridge point (25.31):

- VGG-16, with its massive parameter footprint (747 MB), is expected to generate the highest DRAM traffic and thus the *lowest* AI, predicting memory-bound behavior on all GPUs.
- ViT-B-16, with the most compact memory footprint (337 MB) and highest computational density (17.6 GFLOPs forward), is expected to achieve the *highest* AI, potentially crossing into the compute-bound region.
- ResNet-50 is expected to fall between VGG-16 and ViT in AI, but its large activation footprint (287 MB fwd/bwd) predicts it will remain memory-bound.

These predictions are validated by the empirical measurements in Section C.

C Measurement

Data was collected using NVIDIA Nsight Compute (`ncu`). To isolate the neural network computation from data loading overhead, `torch.cuda.profiler` hooks were injected directly around the forward pass, loss calculation, backward pass, and optimizer step. Measurements were taken on the 11th batch after a 10-batch warm-up phase to avoid cold-start anomalies.

C.1 FLOP Counting Methodology

To determine the placement of each model on the Roofline graph, the theoretical Arithmetic Intensity was calculated using the raw hardware counters extracted via Nsight Compute. The following SASS-level instruction counters were queried:

- `smsp__sass_thread_inst_executed_op_fadd_pred_on.sum` (FADD)
- `smsp__sass_thread_inst_executed_op_fmula_pred_on.sum` (FMUL)
- `smsp__sass_thread_inst_executed_op_ffma_pred_on.sum` (FFMA)
- `smsp__sass_thread_inst_executed_op_hmma_pred_on.sum` (HMMA)
- `dram__bytes_read.sum`, `dram__bytes_write.sum` (DRAM traffic)
- `gpu__time_duration.sum` (kernel wall-clock time)

Since Fused Multiply-Add (FMA) instructions perform two floating-point operations (a multiply and an add) simultaneously, the FFMA counter was doubled. For Tensor Core operations, the HMMA counter was scaled by 512 (reflecting 512 FP16 equivalent operations per HMMA instruction):

$$\text{Total FLOPs} = \text{FADD} + \text{FMUL} + (2 \times \text{FFMA}) + (512 \times \text{HMMA}) \quad (1)$$

$$\text{Arithmetic Intensity} = \frac{\text{Total FLOPs}}{\text{DRAM}_{\text{read}} + \text{DRAM}_{\text{write}}} \quad (2)$$

Important caveat: Since all training runs used FP32 precision (the PyTorch default), the HMMA counter was expected to read zero across all T4 and L4 experiments because Tensor Cores (HMMA) are only engaged for half-precision, mixed-precision, or TF32 workloads. However, on the A100 (Ampere architecture), PyTorch automatically enables TensorFloat-32 (TF32) precision for convolutions and matrix multiplications, routing computation to 3rd-generation Tensor Cores via `mma.sync` instructions. These instructions are *not* captured by the Turing-era `hmma` SASS query, leading to severely undercounted FLOPs on the A100. This profiling artifact is discussed in depth in Section E.

C.2 Empirical Results

Model	Mode	Total FLOPs	DRAM (Bytes)	AI (FLOPs/B)	Perf (GFLOPS/s)
ResNet-50	Train	173.7B	15.7GB	11.06	1,349
ResNet-50	Eval	61.7B	4.56GB	13.54	1,599
VGG-16	Train	347.3B	34.0GB	10.21	1,513
VGG-16	Eval	141.2B	8.88GB	15.90	1,861
ViT-B-16	Train	889.6B	30.7GB	28.96	2,228
ViT-B-16	Eval	298.5B	6.17GB	48.36	2,470

Table 3: Empirical Hardware Measurements on **NVIDIA T4** ($B = 8$). Ridge Point ≈ 25.31 FLOPs/Byte.

Model	Mode	Total FLOPs	DRAM (Bytes)	AI (FLOPs/B)	Perf (GFLOPS/s)
ResNet-50	Train	12.2B	7.61GB	1.60	259
ResNet-50	Eval	6.3B	2.03GB	3.10	468
VGG-16	Train	24.0B	17.9GB	1.34	243
VGG-16	Eval	10.2B	3.76GB	2.72	419
ViT-B-16	Train	829.1B	13.5GB	61.21	4,937
ViT-B-16	Eval	278.4B	1.98GB	140.62	5,862

Table 4: Empirical Hardware Measurements on **NVIDIA L4** ($B = 8$). Ridge Point ≈ 101.0 FLOPs/Byte.

Model	Mode	Total FLOPs*	DRAM (Bytes)	AI (FLOPs/B)	Perf (GFLOPS/s)
ResNet-50	Train	7.1B	7.37GB	0.97	454
ResNet-50	Eval	4.6B	1.93GB	2.41	975
VGG-16	Train	12.6B	16.9GB	0.74	536
VGG-16	Eval	5.2B	3.68GB	1.42	851
ViT-B-16	Train	831.6B	12.9GB	64.62	10,737
ViT-B-16	Eval	280.3B	1.90GB	147.21	11,392

Table 5: Empirical Hardware Measurements on **NVIDIA A100** ($B = 8$). Ridge Point ≈ 12.54 FLOPs/Byte. **CNN FLOP counts are artificially depressed due to Ampere TF32 Tensor Core instructions not captured by Turing-era HMMA SASS queries.*

D Roofline Modeling

Using the empirical data collected in Section C, we constructed Roofline models mapping the theoretical peak performance against the memory bandwidth ceiling for all three GPUs. Each model’s placement on the Roofline graph is determined by its measured Arithmetic Intensity (x-axis) and Attained Performance (y-axis). Points below the ridge point are memory-bound; points to the right are compute-bound. The roofline boundary itself represents the theoretical maximum achievable performance at a given AI.

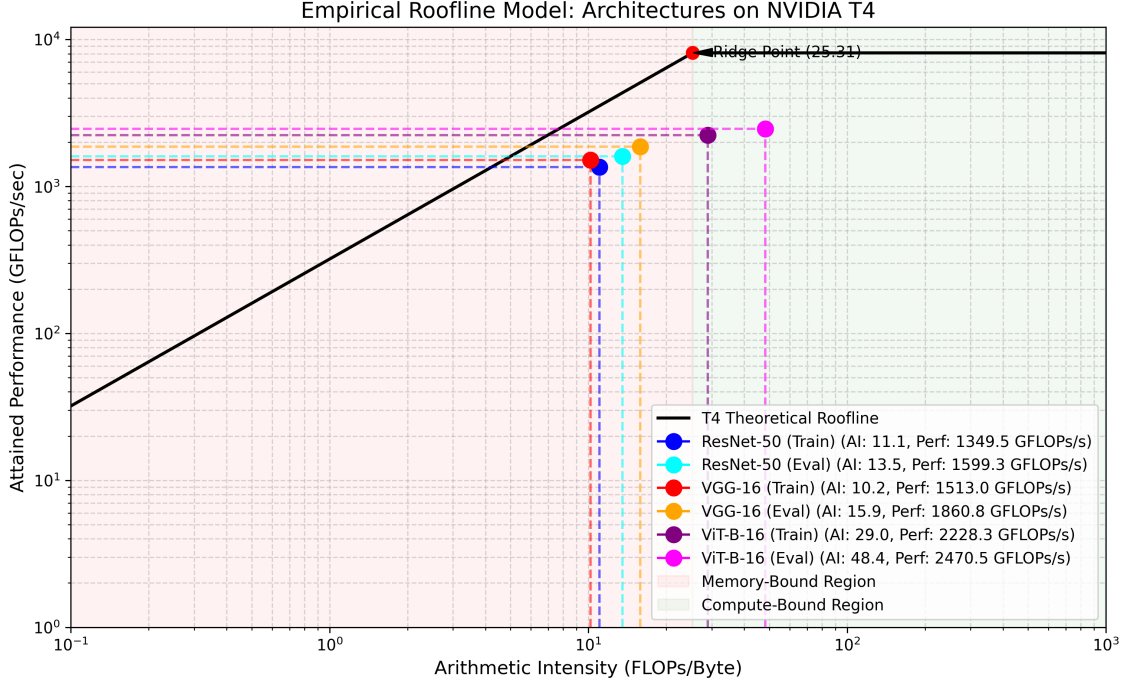


Figure 1: Empirical Roofline Model on **NVIDIA T4** (Turing). Ridge Point = 25.31 FLOPs/Byte. All CNN workloads fall in the memory-bound region. ViT training sits near the ridge; ViT evaluation crosses into compute-bound territory.

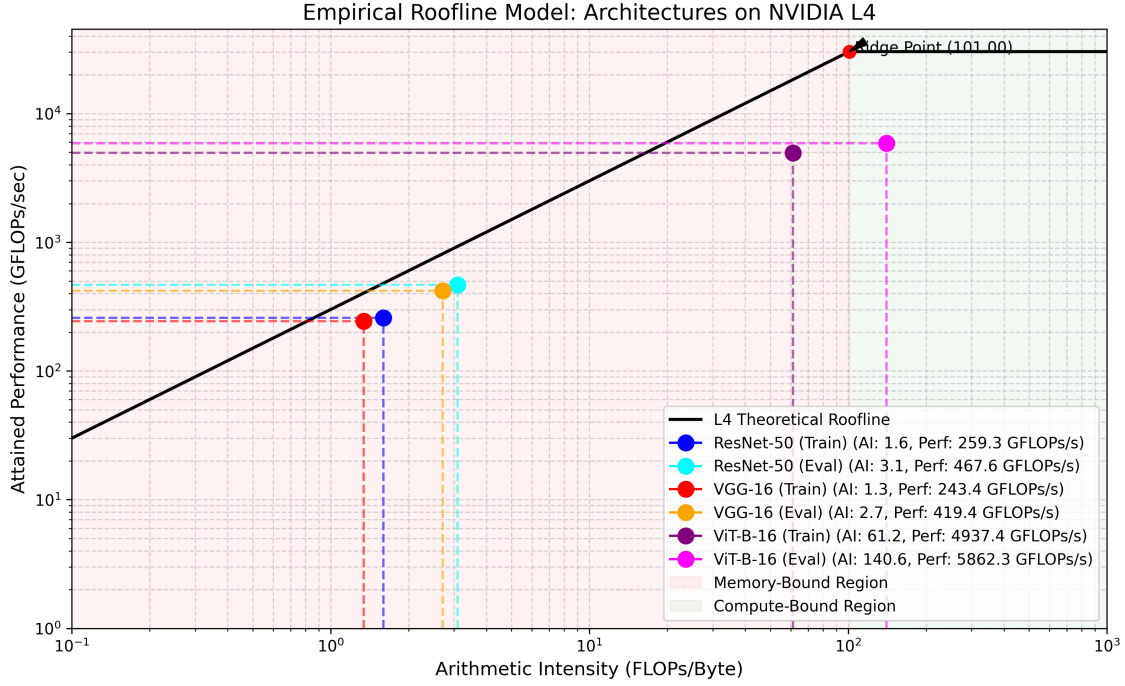


Figure 2: Empirical Roofline Model on **NVIDIA L4** (Ada Lovelace). Ridge Point = 101.0 FLOPs/Byte. The dramatically elevated ridge point means all workloads except ViT evaluation are memory-bound.

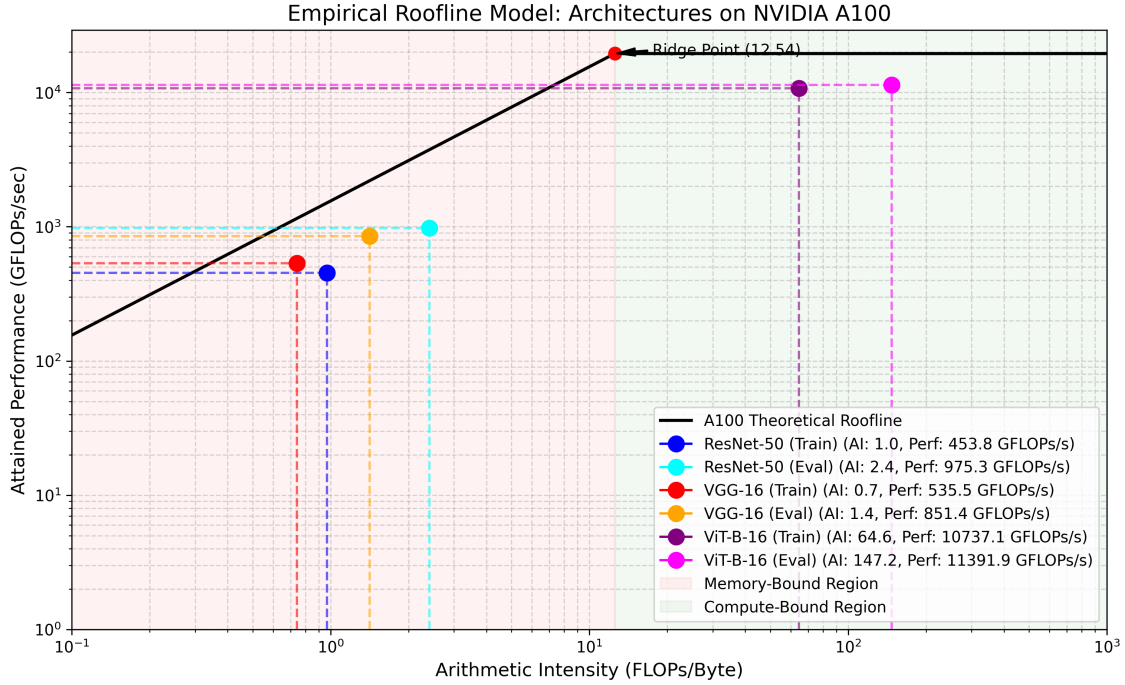


Figure 3: Empirical Roofline Model on **NVIDIA A100** (Ampere). Ridge Point = 12.54 FLOPs/Byte. CNN data points are compressed near AI < 3 due to the TF32 profiling artifact. ViT retains high AI and crosses fully into the compute-bound region.

E Analysis

E.1 Cross-GPU Hardware Comparison

The three GPUs in this study represent fundamentally different compute/bandwidth trade-offs, as summarized in Table 6.

GPU	FP32 Peak (TFLOPS)	BW (GB/s)	Ridge (FLOPs/B)	Architecture
T4	8.1	320	25.31	Turing (2018)
A100	19.5	1,555	12.54	Ampere (2020)
L4	30.3	300	101.0	Ada Lovelace (2023)

Table 6: GPU Hardware Specifications and Ridge Points

These ridge points carry significant implications:

- The **A100** has the *lowest* ridge point (12.54) because its HBM2e memory delivers 1,555 GB/s, nearly $5\times$ the bandwidth of the T4 and L4. This means workloads need a much lower AI to escape the memory-bound region. The A100 is architected as a *bandwidth-rich* accelerator.
- The **L4** has the *highest* ridge point (101.0) because it pairs very high FP32 compute (30.3 TFLOPS, boosted by Ada Lovelace Tensor Cores for TF32) with only 300 GB/s of GDDR6 bandwidth. This means almost all workloads are memory-bound on the L4 unless they achieve extremely high arithmetic intensity. The L4 is architected as a *compute-rich, bandwidth-starved* inference accelerator.
- The **T4** sits in the middle (25.31), with a balanced but modest profile that places CNN workloads in the memory-bound region and ViT workloads near the transition zone.

E.2 Influence of the Environment and Profiling Constraints

E.2.1 The TF32 Profiling Artifact on the A100

The transition from the Turing (T4) to the Ampere (A100) architecture highlights a critical challenge in hardware-aware profiling. On the T4, all architectures registered Arithmetic Intensities between 10.2 and 48.4 FLOPs/Byte, yielding easily interpretable roofline placements. However, on the A100, the calculated AI for *CNN models* plummeted dramatically: ResNet-50 training dropped from 11.06 (T4) to 0.97 (A100), and VGG-16 training dropped from 10.21 to 0.74.

This drastic drop is **not** indicative of the neural networks performing fewer operations. Rather, it is a **profiling artifact**. PyTorch automatically leverages TensorFloat-32 (TF32) precision on Ampere Tensor Cores for `torch.nn.Conv2d` and `torch.nn.Linear` operations. Because our Nsight Compute methodology queried scalar SASS instructions (`fadd/fmul/ffma`) and Turing-specific `mma` instructions, it remained “blind” to the 3rd-generation `mma.sync` Tensor Core operations executing the bulk of the convolution and GEMM (General Matrix Multiply) workload. Consequently:

- The A100’s CNN **FLOP counts were severely undercounted** (e.g., ResNet-50 training: 173.7B on T4 vs. 7.1B on A100 – a $24\times$ reduction).

- **Attained Performance was artificially depressed**, dragging the empirical data points down the Y-axis.
- **Arithmetic Intensity collapsed below 1.0** for CNN training, placing them in an extremely memory-bound position that does not reflect the true computational character of the workload.

This finding is independently corroborated by comparing **ViT results** across GPUs. ViT’s measured FLOPs remain remarkably consistent ($\sim 830\text{--}890\text{B}$ for training across all three GPUs), while CNN FLOPs vary by an order of magnitude. This requires careful interpretation: ViT’s `nn.Linear` layers (Q/K/V projections, MLP blocks) *also* route to TF32 Tensor Cores on the A100, so one might expect ViT FLOPs to be similarly undercounted. The apparent consistency arises because a substantial fraction of ViT’s total captured FLOPs comes from operations that remain on CUDA cores, `softmax` computations ($O(n^2)$ per head per layer), layer normalization, residual additions, and GELU activations, which are captured by our FADD/FMUL/FFMA queries. These CUDA-core operations serve as a “floor” that dominates the captured FLOP count even when the TF32-routed matmul FLOPs are missed. In contrast, CNNs consist almost entirely of convolution and FC layers (both TF32-eligible), with minimal scalar operations, leaving almost no captured FLOPs when the Tensor Core operations go uncounted.

E.2.2 Evidence from DRAM Consistency

A critical validation of this interpretation comes from the **DRAM byte counts**, which are accurately measured regardless of which compute units execute the FLOPs. The DRAM traffic for each model remains remarkably consistent across GPUs:

- ResNet-50 Training DRAM: 15.7 GB (T4) vs. 7.6 GB (L4) vs. 7.4 GB (A100)
- VGG-16 Training DRAM: 34.0 GB (T4) vs. 17.9 GB (L4) vs. 16.9 GB (A100)
- ViT-B-16 Training DRAM: 30.7 GB (T4) vs. 13.5 GB (L4) vs. 12.9 GB (A100)

The T4’s higher DRAM counts may reflect Turing’s less efficient memory access patterns. The L4 and A100 DRAM counts are closely aligned, confirming that the algorithms are performing the same fundamental work despite the apparent FLOP discrepancy.

E.2.3 HMMA = 0: Expected Behavior

Across all experiments, the HMMA instruction counter reads exactly zero. This is **expected** for two reasons:

1. On the **T4 and L4**, training in FP32 uses CUDA cores (`ffma` instructions), not Tensor Cores (`hmma`). Tensor Cores on Turing and Ada Lovelace require explicit mixed-precision or FP16 inputs.
2. On the **A100**, although Tensor Cores *are* active via TF32 mode, they use the Ampere-specific `mma.sync` instruction—a different ISA opcode from the Turing `hmma` instruction that our query targets. The `hmma` counter therefore correctly reads zero; the issue is that the query is asking for the wrong instruction.

E.3 Influence of the Neural Network Architecture

E.3.1 CNNs are Consistently Memory-Bound

Despite the profiling limitations on the A100, the empirical results show a consistent pattern: **CNN workloads (ResNet-50 and VGG-16) are memory-bound across all three GPUs.**

On the T4, ResNet-50 and VGG-16 registered AIs between 10.2–15.9 FLOPs/Byte, well below the ridge point of 25.31. On the L4, their AIs further dropped to 1.3–3.1 FLOPs/Byte against a ridge of 101.0. On the A100, measured AIs were 0.7–2.4 FLOPs/Byte (noting the TF32 undercounting), but even correcting for the artifact, their true AI would likely remain below the A100’s ridge point of 12.54.

This memory-bound behavior is fundamentally driven by the CNN computational pattern:

- 3×3 convolutions have small reuse factors and require streaming large activation tensors through memory.
- VGG-16’s three fully-connected layers ($25,088 \rightarrow 4,096 \rightarrow 4,096 \rightarrow 1,000$) generate enormous parameter traffic, explaining why VGG-16 consistently has the *highest DRAM traffic* (17–34 GB for training) and *lowest AI* across all environments.
- ResNet-50’s skip connections require storing additional intermediate activations for the backward pass, inflating memory access relative to compute.

E.3.2 ViT is Compute-Bound (or Near the Transition)

The Vision Transformer exhibits fundamentally different behavior. On the T4, ViT-B-16 training achieves an AI of 28.96 FLOPs/Byte, just above the ridge point (25.31), placing it in the *compute-bound* region. In evaluation mode, its AI jumps to 48.36, firmly compute-bound.

On the L4, ViT training (AI = 61.21) and evaluation (AI = 140.62) remain well above the CNN data but are still memory-bound relative to the L4’s extreme ridge point of 101.0. Only ViT evaluation crosses the L4’s ridge. On the A100, ViT’s AI reaches 64.62 (train) and 147.21 (eval), both well above the A100’s modest ridge of 12.54, confirming compute-bound behavior.

This compute-bound tendency stems from the transformer architecture:

- The **multi-head self-attention** mechanism computes $Q \cdot K^T$ and then $(Q \cdot K^T) \cdot V$ projections, which are large, dense matrix multiplications with high data reuse.
- The **MLP blocks** in each encoder layer consist of two linear projections ($768 \rightarrow 3072 \rightarrow 768$) that are also compute-dense.
- The **patch embedding** reduces spatial dimensions early (from 224×224 pixels to $14 \times 14 = 196$ patches), minimizing the total memory footprint of intermediate activations.

E.3.3 Training vs. Evaluation: The Backward Pass Effect

Across all models and GPUs, **evaluation mode consistently yields higher Arithmetic Intensity** than training mode. This pattern is universal:

- T4: ResNet-50 AI increases from 11.06 (train) to 13.54 (eval), VGG-16 from 10.21 to 15.90, ViT from 28.96 to 48.36.

- L4: ResNet-50 AI increases from 1.60 to 3.10, VGG-16 from 1.34 to 2.72, ViT from 61.21 to 140.62.
- A100: ResNet-50 AI increases from 0.97 to 2.41, VGG-16 from 0.74 to 1.42, ViT from 64.62 to 147.21.

This shift is fundamentally driven by the backward pass’s heavy memory footprint. During training:

1. The framework must **store intermediate activation maps** at each layer to compute gradients during backpropagation. These activations drastically increase DRAM read/write volume.
2. The **gradient computation and optimizer step** generate additional memory traffic (reading parameters, writing gradient buffers, updating weights) without a proportional increase in FLOPs.
3. The net effect is that the denominator of the AI equation (Eq. 2) grows much faster than the numerator, pushing training runs deeper into the memory-bound region.

For ViT specifically, the AI roughly doubles between training and evaluation across all GPUs. This is because the attention operation’s $O(n^2 \cdot d)$ compute remains dense in forward-only mode, while the memory overhead of storing attention weight matrices for backpropagation is eliminated during evaluation.

E.4 Performance Scaling Across GPUs

Examining the attained performance across GPUs reveals important scaling characteristics:

- **CNN performance scales with bandwidth:** ResNet-50 training attains 1,349 GFLOPS/s on T4, 259 on L4, and 454 on A100 (noting the A100 value is undercounted). The L4’s lower raw attained performance despite higher peak compute confirms that CNNs are bandwidth-limited: the L4’s 300 GB/s bandwidth is the bottleneck, not its 30.3 TFLOPS of compute.
- **ViT performance scales with compute:** ViT training attains 2,228 GFLOPS/s on T4, 4,937 on L4, and 10,737 on A100. The A100 achieves $4.8\times$ the T4’s performance despite only a $2.4\times$ FP32 peak compute ratio (19.5 vs. 8.1 TFLOPS). This $2\times$ “excess” speedup is explained by the A100’s TF32 mode: although our FLOP counters capture similar scalar instruction counts across GPUs, the A100’s Tensor Cores execute the dominant matmul operations at effectively $\sim 2\times$ the FP32 throughput via TF32 precision, providing an additional performance multiplier that is invisible to our SASS queries but manifests as faster wall-clock kernel execution times. Thus, $2.4\times$ (compute ratio) $\times \sim 2\times$ (TF32 acceleration) $\approx 4.8\times$ observed speedup.
- **The L4 paradox:** Despite being the newest GPU (2023) with the highest FP32 peak (30.3 TFLOPS), the L4 delivers the *lowest* CNN performance among the three GPUs. This occurs because its GDDR6 bandwidth (300 GB/s) is the bottleneck for memory-bound workloads. For ViT, the L4 performs $2.2\times$ better than the T4, demonstrating that its compute advantages are only realized for compute-bound workloads. This makes the L4 optimally suited for its intended use case: inference of compute-dense models.

E.5 Hardware Utilization Efficiency

To assess how effectively each workload exploits the available hardware, we compute the *hardware utilization efficiency* as the ratio of attained performance to the theoretical peak:

$$\eta = \frac{\text{Attained Performance (GFLOPS/s)}}{\text{Theoretical Peak (GFLOPS/s)}} \times 100\%$$

Model (Mode)	T4 η	L4 η	A100 η
ResNet-50 (Train)	16.7%	0.9%	2.3%*
ResNet-50 (Eval)	19.7%	1.5%	5.0%*
VGG-16 (Train)	18.7%	0.8%	2.7%*
VGG-16 (Eval)	23.0%	1.4%	4.4%*
ViT-B-16 (Train)	27.5%	16.3%	55.1%
ViT-B-16 (Eval)	30.5%	19.3%	58.4%

Table 7: Hardware Utilization Efficiency (η). *A100 CNN values are artificially depressed due to the TF32 FLOP undercounting artifact.

Several insights emerge:

- **ViT on A100 achieves ~55–58% utilization**, a meaningfully high figure for a real-world training workload, indicating that the transformer’s dense matmul pattern is well-matched to the A100’s Tensor Core architecture.
- **T4 utilization is modest but consistent** (17–30%), reflecting the T4’s balanced architecture where neither compute nor bandwidth is dramatically over-provisioned.
- **L4 CNN utilization is extremely low** (<2%), which is expected: memory-bound workloads on a bandwidth-starved GPU cannot exercise the available compute, leaving the L4’s 30.3 TFLOPS almost entirely idle during CNN training. This quantitatively confirms the “L4 paradox” and underscores that raw TFLOPS is a misleading performance metric for memory-bound workloads.
- The >50% utilization for ViT on A100 suggests that further optimization (mixed-precision training, larger batch sizes, operator fusion) could push utilization even higher, potentially approaching the 60–70% range typically seen in optimized training pipelines.

E.6 Summary of Key Findings

1. **Architecture determines bottleneck character:** CNNs (ResNet-50, VGG-16) are consistently memory-bound across all GPUs due to their small-kernel convolution patterns and large activation footprints. ViT (ViT-B-16) is consistently compute-bound (or near the transition) due to its dense matrix multiplications in self-attention and MLP blocks.
2. **Hardware determines bottleneck severity:** The L4’s extreme ridge point (101.0) makes almost all workloads memory-bound, while the A100’s low ridge point (12.54) allows even moderately compute-intensive workloads to become compute-bound.

3. **Cross-generational profiling requires ISA-aware instrumentation:** Turing-era SASS queries (`fadd/fmul/ffma/hmma`) are blind to Ampere’s `mma.sync` Tensor Core instructions, producing severely undercounted FLOP measurements. Future work should use architecture-specific metrics or NVIDIA’s built-in Roofline analysis in Nsight Compute to avoid this artifact.
4. **Training is more memory-bound than evaluation:** The backward pass universally increases DRAM traffic relative to FLOPs, reducing AI by $1.2\text{--}2.3\times$ compared to forward-only evaluation.
5. **VGG-16’s fully-connected layers dominate memory traffic:** Despite generating the most FLOPs (347B on T4), VGG-16’s AI is consistently the lowest among the three models, driven by the massive parameter traffic of its FC layers.